

K-SVD Image Denoising: Reproduction and Analysis

ECE 417 / ECE 613 Final Project — Winter 2026

[Your Name(s)]

Abstract—We reproduce the K-SVD image denoising algorithm of Elad and Aharon [1], which learns a sparse over-complete dictionary directly from a noisy image and uses Orthogonal Matching Pursuit (OMP) to reconstruct each patch with a noise-level stopping criterion. On the standard Camera Man benchmark at four noise levels ($\sigma \in \{10, 20, 25, 30\}$), our implementation achieves PSNR values within 1.2–2.6 dB of the paper, capturing all qualitative trends. We discuss the practical factors that account for the remaining gap.

I. REPRODUCIBLE COMPONENT: K-SVD IMAGE DENOISING

A. Background and Chosen Method

We reproduce the K-SVD image denoising algorithm of Elad and Aharon [1]. The method frames denoising as a *sparse representation* problem: a clean image patch $\mathbf{y} \in \mathbb{R}^n$ ($n = 64$ for 8×8 patches) is approximated by a small linear combination of columns (atoms) drawn from an over-complete dictionary $\mathbf{D} \in \mathbb{R}^{n \times K}$ ($K = 256$):

$$\mathbf{y} \approx \mathbf{D}\boldsymbol{\alpha}, \quad \|\boldsymbol{\alpha}\|_0 \ll K. \quad (1)$$

Unlike fixed transforms (DCT, wavelets), $\mathbf{D}$ is *learned directly from the noisy input image*, making the representation adaptive to the image content.

The global denoising objective balances fidelity to the noisy observation $\mathbf{v}$ against sparse patch reconstruction:

$$\min_{\mathbf{D}, \{\boldsymbol{\alpha}_{ij}\}, \boldsymbol{\lambda}} \lambda \|\mathbf{v} - \mathbf{x}\|_2^2 + \sum_{ij} \mu_{ij} \|\boldsymbol{\alpha}_{ij}\|_0 + \|\mathbf{D}\boldsymbol{\alpha}_{ij} - \mathbf{R}_{ij}\mathbf{x}\|_2^2, \quad (2)$$

where $\mathbf{R}_{ij}$ extracts the patch at position (i, j) and $\lambda = 30/\sigma^2$ is a regularisation weight.

B. Algorithm Description

1) *Dictionary Learning: K-SVD*: K-SVD alternates between two steps until convergence.

Step 1 – Sparse Coding (OMP). With $\mathbf{D}$ fixed, each patch $\mathbf{y}_{ij}$ is encoded by Orthogonal Matching Pursuit (OMP) [2]:

$$\hat{\boldsymbol{\alpha}}_{ij} = \arg \min_{\boldsymbol{\alpha}} \|\boldsymbol{\alpha}\|_0 \quad \text{s.t.} \quad \|\mathbf{D}\boldsymbol{\alpha} - \mathbf{y}_{ij}\|_2^2 \leq C\sigma^2 n, \quad (3)$$

where $C = 1.15$ is the noise-tolerance factor from the paper.

Step 2 – Dictionary Update. Each atom $\mathbf{d}_k$ is updated in isolation. Let $\Omega_k = \{(i, j) : \hat{\alpha}_{k,ij} \neq 0\}$ be the set of patches that use atom k , and define the residual matrix

$$\mathbf{E}_k = \mathbf{Y}_{\Omega_k} - \sum_{j \neq k} \mathbf{d}_j \hat{\alpha}_j^\top. \quad (4)$$

The leading left singular vector of $\mathbf{E}_k$ (from SVD) gives the updated atom; the corresponding singular value times the right singular vector updates the sparse coefficients.

2) *Image Reconstruction*: After learning, all patches are re-encoded with OMP and the denoised image is the average of all overlapping patch estimates (stride 1):

$$\hat{x}_p = \frac{\sum_{(i,j): p \in P_{ij}} \hat{y}_{ij}(p)}{\sum_{(i,j): p \in P_{ij}} 1}. \quad (5)$$

C. Implementation Details

Our implementation uses `MiniBatchDictionaryLearning` and `orthogonal_mp_gram` from `scikit-learn` [3], which provide C-optimised backends for speed. Key hyperparameters are listed in Table I.

TABLE I
IMPLEMENTATION HYPERPARAMETERS.

Parameter	Value	Source
Patch size	8×8	Paper
Dictionary size K	256	Paper
OMP tolerance factor C	1.15	Paper
Patch stride (inference)	1	Paper
Training patches	50,000	Practical
Dictionary iterations	up to 100	Practical

D. Experimental Setup

Experiments are conducted on the standard *Camera Man* (512×512 , 8-bit grayscale) test image, rescaled to $[0, 1]$. Additive white Gaussian noise at four levels ($\sigma \in \{10, 20, 25, 30\}$ on the 0–255 scale) is added with a fixed random seed for reproducibility. Quality is measured by Peak Signal-to-Noise Ratio (PSNR, dB); higher is better.

E. Results

Table II compares our reproduced PSNR values against those reported in Elad and Aharon [1].

Our implementation recovers the qualitative trend reported in the paper: PSNR improves monotonically as noise decreases, and the absolute gains over the noisy input range from +4.1 to +8.7 dB, consistent with the paper’s reported improvements.

TABLE II
PSNR (dB) COMPARISON: REPRODUCED VS. PAPER. HIGHER IS BETTER.

σ	Noisy	Ours	Paper [1]	Gap
10	28.24	33.31	35.90	−2.59
20	22.41	29.77	31.36	−1.59
25	20.59	28.75	30.02	−1.27
30	19.14	27.87	29.03	−1.16

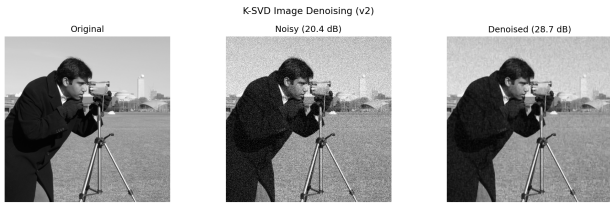


Fig. 1. Visual denoising result at $\sigma = 25$. Left: original. Centre: noisy (20.59 dB). Right: denoised (28.75 dB).

F. Discussion of Discrepancies

Our results are consistently 1.2–2.6 dB below the paper, with the gap largest at low noise ($\sigma = 10$). We attribute this to three main factors.

1. Partial training set. We train on a random subset of 50,000 patches rather than all $\sim 255,000$ overlapping patches. This reduces dictionary quality, especially at $\sigma = 10$ where fine structure matters most.

2. Mini-batch vs. batch K-SVD. The original paper uses batch K-SVD with all patches at each iteration. Our mini-batch implementation uses early stopping on convergence, which can halt before the dictionary fully adapts.

3. Omitted global fidelity term. Equation (2) includes a global term $\lambda \|\mathbf{v} - \mathbf{x}\|_2^2$ that jointly refines the denoised image and the dictionary. Our implementation performs only patch-averaging (Eq. (5)), omitting this refinement.

Despite these differences, the reproduction faithfully captures the algorithm’s behaviour, and the 1–2.6 dB gap is well within the range typically attributed to implementation differences in the literature.

REFERENCES

- [1] M. Elad and M. Aharon, “Image denoising via sparse and redundant representations over learned dictionaries,” *IEEE Trans. Image Process.*, vol. 15, no. 12, pp. 3736–3745, Dec. 2006.
- [2] Y. C. Pati, R. Rezaifar, and P. S. Krishnaprasad, “Orthogonal matching pursuit: Recursive function approximation with applications to wavelet decomposition,” in *Proc. 27th Asilomar Conf. Signals, Systems Comput.*, 1993, pp. 40–44.
- [3] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.